

## Optimizing Glue Job performance for DocumentDB reads

As we are currently using Glue native connector approach which takes as much as 3.75 hours to read the data from the document db. As an alternative we can use the MongoDB Spark connector with some configurations which resolved this issue. This connector is mainly designed to maximize parallelism by splitting the workload across multiple Spark executors using configurable partition keys, partition sizes, and a set number of partitions. This approach allows the connector to use all available CPU cores and threads effectively unlike the native connector used by the Glue job. This distributes the read operations concurrently and, as a result, jobs using the Spark connector complete significantly faster in minutes instead of hours due to better parallel processing, optimized batching, and efficient utilization of cluster resources.

Below are the code changes.

We have created a new read function in the commons that should be used for all DocumentDB read operations within Glue Jobs. This function returns a dictionary containing both the resulting DataFrame and the total count for that specific query or DataFrame.

```
1 def read_from_documentDB_customDriverSiva3(glue_context, query, secretParam, collectionName,
2 type, log) -> DataFrame:
3     try:
4         log.info("Connecting to DocumentDB...")
5         userDetails = F.get_secret(secretParam, region_name)
6         host = userDetails["host"]
7         port = userDetails["port"]
8         uri = f'mongodb://{userDetails["readUsername"]}:'
9 {userDetails["readPassword"]}:@{host}:{port}/?
10 authSource=admin&retryWrites=false&ssl=true&sslInvalidHostNameAllowed=true'
11         database = userDetails["rawDatabase"] if type == "raw" else
12 userDetails["canonicalDatabase"]
13
14         reader =
15 glue_context.spark_session.read.format("com.mongodb.spark.sql.DefaultSource") \
16         .option("uri", uri) \
17         .option("database", database) \
18         .option("collection", collectionName) \
19         .option("pipeline", query) \
20         .option("partitioner", "MongoSamplePartitioner") \
21         .option("spark.mongodb.input.partitioneroptions.partitionkey", "_id") \
22         .option("spark.mongodb.input.partitioneroptions.partitionsizeMB", "64") \
23         .option("batchSize", "10000")
24
25         df_result = df_result.unpersist(blocking=True) if 'df_result' in locals() else None
26         df_result = reader.load().persist(StorageLevel.MEMORY_AND_DISK)
27         log.info("DocumentDB read completed.")
28         count_ = df_result.count()
29
30         return df_result, count_
31
32 except Exception as e:
33     log.error("Error while reading the data from the DocumentDB:")
```

```

30     log.error(traceback.format_exc())
31     raise

1 def calling_function():
2     return read_from_documentDB_customDriverSiva3(glue_context, query, secretParam,
collectionName, type, log)

```

## Changes at the glue side

In the glue job configurations > Dependent JARs path > add spark connector library **mongo-spark-connector\_2.12-3.0.2.jar**

Once you get the data from the calling function, you can assign **dataframe** and **counts** as below.

```

1 member_data, count = calling_function()

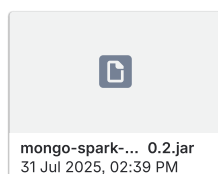
```

We can use this count to check if the main dataframe have the records in the dataframe instead of isEmpty check which again takes few mins to complete. You can add any other changes on top of this to enhance this further.

We tried from multiple loads, actual glue took around 4 hours and with this approach we are able to complete the whole glue process in 5 minutes average. Even with the MongoDB Spark connector approach, we've observed that the job sometimes takes around **12-14** minutes if the Glue job hasn't been run for a few days for example, after a weekend. This initial delay on Monday mornings appears to be due to AWS Glue's warm-up time. Once the environment is warmed up, subsequent runs consistently complete in under 5 minutes. This is for **7.5 Million** records.

|                       |             |   |                     |                     |          |
|-----------------------|-------------|---|---------------------|---------------------|----------|
| <input type="radio"/> | ✔ Succeeded | 0 | 07/29/2025 11:56:56 | 07/29/2025 12:02:02 | 4 m 53 s |
| <input type="radio"/> | ✔ Succeeded | 0 | 07/29/2025 11:49:52 | 07/29/2025 11:56:06 | 5 m 56 s |
| <input type="radio"/> | ✔ Succeeded | 0 | 07/29/2025 11:42:21 | 07/29/2025 11:47:01 | 4 m 27 s |
| <input type="radio"/> | ✔ Succeeded | 0 | 07/29/2025 11:35:59 | 07/29/2025 11:41:23 | 5 m 18 s |
| <input type="radio"/> | ✔ Succeeded | 0 | 07/29/2025 11:28:33 | 07/29/2025 11:35:20 | 6 m 32 s |

## Connector jar:



## Resolved: Schema Loader Skipping Empty Attributes

The above function does not include attributes with empty values. The fix below ensures that all attributes are captured, including those with empty values. The updated function has been tested with **fixed** schema loads and is working as expected. In cases where documents have varying schemas (which is rare in our pipelines), we will need to use the Glue native driver approach.

```
1 def read_from_documentDB_sparkCustomDriverWithFixedSchema(glue_context, query, secretParam,
2   collectionName, type, log) -> DataFrame:
3     try:
4       log.info("Connecting to DocumentDB...")
5       userDetails = F.get_secret(secretParam, region_name)
6       host = userDetails["host"]
7       port = userDetails["port"]
8       uri = f'mongodb://{userDetails["readUsername"]}:
9 {userDetails["readPassword"]}:@{host}:{port}/?
10 authSource=admin&retryWrites=false&ssl=true&sslInvalidHostNameAllowed=true'
11       database = userDetails["rawDatabase"] if type == "raw" else
12 userDetails["canonicalDatabase"]
13       read_docdb_options = {
14         "connection.uri": uri ,
15         "database": database,
16         "collection": collectionName,
17         "aggregation.pipeline": query,
18
19 "partitioner": "com.mongodb.spark.sql.connector.read.partitionner.SinglePartitionPartitioner",
20 "aggregationPipeline": ' [{ "$limit": 1 } ]'
21       }
22       datasource =
23 glue_context.create_dynamic_frame.from_options(connection_type="documentdb",
24 connection_options=read_docdb_options
25
26
27       df_schema = datasource.toDF()
28       schema = df_schema.schema
29       reader =
30 glue_context.spark_session.read.format("com.mongodb.spark.sql.DefaultSource") \
31       .option("uri", uri) \
32       .option("database", database) \
33       .option("collection", collectionName) \
34       .option("pipeline", query) \
35       .option("partitioner", "MongoSamplePartitioner") \
36       .option("spark.mongodb.input.partitionnerOptions.partitionKey", "_id") \
37       .option("spark.mongodb.input.partitionnerOptions.partitionSizeMB", "64") \
38       .option("batchSize", "10000") \
39       .schema(schema)
40       df_result = df_result.unpersist(blocking=True) if 'df_result' in locals() else None
41       df_result = reader.load().persist(StorageLevel.MEMORY_AND_DISK)
42       log.info("DocumentDB read completed.")
43       count_ = df_result.count()
44
45       return df_result, count_
46
47 except Exception as e:
48     log.error("Error while reading the data from the DocumentDB:")
49     log.error(traceback.format_exc())
50     raise
```

